Machine Learning Revision Notes

Beginner to Intermediate Level

For Exam Revision | Concept Building | RAG Projects

7 Core Topics • Clean Summaries • RAG-Ready Structure

Table of Contents

Section 1 | Introduction to Machine Learning

What is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence where computers learn from data to make decisions or predictions — **without being explicitly programmed** for every task.

Think of it like teaching a child:

- You show examples (data)
- The child finds patterns
- Later it can answer new questions based on those patterns

> ■ *Note: ML works best when there is a lot of quality data available.*

Types of Machine Learning

Type	How It Learns	Real-World Example
Supervised Learning	Learns from labeled data (input + correct output)	Spam email detection, house price prediction
Unsupervised Learning	Finds hidden patterns in unlabeled data	Customer segmentation, topic grouping
Reinforcement Learning	Learns by trial-and-error using rewards	Game playing (Chess, Go), robotics

Quick Summary

- **Supervised**: Has labels — knows the answer during training
- **Unsupervised**: No labels — finds patterns on its own
- **Reinforcement**: Agent learns through rewards and penalties

Section 2 | Data Preprocessing

Before training a model, raw data must be **cleaned and prepared**. Good preprocessing directly improves model accuracy.

1. Handling Missing Values

Missing values are empty cells in your dataset. Common strategies:

- **Remove rows/columns** — Use when missing data is very small (<5%)
- **Mean / Median imputation** — Fill numeric columns with their average or median
- **Mode imputation** — Fill categorical columns with the most frequent value
- **Forward / Backward fill** — Useful in time-series data

> ■ *Note: Use median instead of mean when the column has many outliers.*

2. Feature Scaling

Many ML algorithms are sensitive to the scale (magnitude) of features. Scaling brings all features to a comparable range.

Min-Max Normalization

Scales values to a range of 0 to 1.

```
X_scaled = (X - X_min) / (X_max - X_min)
```

- Output range: [0, 1]
- Use when distribution is NOT Gaussian (normal)
- Sensitive to outliers

Standardization (Z-score)

Centers data around mean=0 with standard deviation=1.

```
X_scaled = (X - mean) / standard_deviation
```

- Output: no fixed range, but values cluster around 0
- Use when distribution IS Gaussian
- Less affected by outliers

Method	Range	Best For
Min-Max Normalization	0 to 1	Neural networks, KNN

Standardization	No fixed range	SVM, Linear Regression, PCA

3. Train-Test Split

We split the dataset into two parts: one for training the model and one for testing how well it generalizes to new, unseen data.

- **Training Set**: Used to fit/train the model (typically 70–80%)
- **Test Set**: Used to evaluate model performance (typically 20–30%)
- Never let the model see test data during training — it defeats the purpose!

■ *Tip: Use random_state parameter in sklearn to get reproducible splits.*

Common Split Ratios

• 80% train / 20% test — most common

• 70% train / 30% test — when data is limited

• 60% train / 20% val / 20% test — for deep learning with validation

Section 3 | Supervised Learning

In supervised learning, the model trains on labeled data — each input has a corresponding correct output. The model learns the mapping between inputs and outputs.

Linear Regression

Used to predict a **continuous numeric value** (e.g., house price, stock price).

- Draws the best-fit straight line through the data
- Output: any real number (e.g., 50000, 3.7, -12)
- Assumes a linear relationship between input and output

```
y = mx + b (slope * input + intercept)
```

■ *Note: Use Mean Squared Error (MSE) to measure how far predictions are from the truth.*

Logistic Regression

Despite the name, it is used for **classification**, not regression.

- Predicts probability of belonging to a class
- Output: a value between 0 and 1 (via sigmoid function)
- If probability > 0.5 → Class 1; otherwise → Class 0
- Example: Is this email spam (1) or not spam (0)?

```
sigmoid(z) = 1 / (1 + e^(-z))
```

■ *Note: Logistic Regression is great for binary classification problems.*

Decision Trees

A Decision Tree splits data into branches based on feature values, like a flowchart of yes/no questions, until it reaches a prediction.

- **Root Node**: The starting question (best feature to split on)
- **Internal Nodes**: More questions / conditions
- **Leaf Nodes**: Final predictions
- Can handle both classification and regression
- Easy to visualize and interpret

■ *Tip: Decision trees tend to overfit. Control depth using max_depth parameter.*

Random Forest

A Random Forest builds **many decision trees** and combines their results. This ensemble approach reduces overfitting and improves accuracy.

- Combines 100s of decision trees (called an ensemble)
- Each tree is trained on a random subset of data and features
- Final output: majority vote (classification) or average (regression)
- More powerful and robust than a single decision tree
- Less interpretable but much more accurate

Why Random Forest Works Well

• Diverse trees reduce variance (less overfitting)
• Handles missing values and outliers well
• Works on both classification and regression
• One of the most popular algorithms in practice

Algorithm	Task Type	Key Strength
Linear Regression	Regression	Simple, interpretable
Logistic Regression	Binary Classification	Probabilistic output
Decision Tree	Both	Visual, easy to explain
Random Forest	Both	High accuracy, robust

Section 4 | Unsupervised Learning

In unsupervised learning, the data has **no labels**. The model finds hidden patterns or groupings on its own.

K-Means Clustering

K-Means groups data points into **K clusters** based on similarity. Points in the same cluster are close to each other.

How it works — Step by Step:

Step 1	Choose K (number of clusters)
Step 2	Randomly place K centroids (center points)
Step 3	Assign each data point to the nearest centroid
Step 4	Move each centroid to the average of its assigned points
Step 5	Repeat steps 3-4 until centroids stop moving

- **K** is a hyperparameter — you must choose it before training
- Use the **Elbow Method** to find the best K
- Applications: customer segmentation, document grouping, image compression

■ *Tip: Plot inertia (sum of squared distances) vs K. The 'elbow' point is the best K.*

PCA — Principal Component Analysis (Basic Intuition)

PCA is a technique to **reduce the number of features** (dimensions) in a dataset while keeping as much important information as possible.

Think of it like this:

- You have 50 features but many of them carry overlapping information
- PCA finds the most important 'directions' in the data
- It compresses those 50 features into 2 or 3 key components
- You lose some detail, but gain speed and simplicity

■ *Note: PCA is mainly used for visualization, noise reduction, and speeding up training.*

When to Use PCA

- Dataset has too many features (high dimensionality)
- Want to visualize high-dimensional data in 2D or 3D
- Model is slow due to too many features
- Features are highly correlated with each other

Section 5 | Model Evaluation

After training a model, we need to measure how well it performs. Different metrics tell us different things about model quality.

Confusion Matrix

A confusion matrix shows the breakdown of correct and incorrect predictions for a classification model.

	Predicted: Positive	Predicted: Negative
Actual: Positive	**TP (True Positive)**	**FN (False Negative)**
Actual: Negative	**FP (False Positive)**	**TN (True Negative)**

- **TP** (True Positive): Correctly predicted Positive
- **TN** (True Negative): Correctly predicted Negative
- **FP** (False Positive): Predicted Positive but actually Negative (Type I Error)
- **FN** (False Negative): Predicted Negative but actually Positive (Type II Error)

Evaluation Metrics

Accuracy

Out of all predictions, how many were correct?

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

Precision

Out of all positive predictions, how many were actually positive?

```
Precision = TP / (TP + FP)
```

Recall (Sensitivity)

Out of all actual positives, how many did we correctly identify?

```
Recall = TP / (TP + FN)
```

F1 Score

Harmonic mean of Precision and Recall. Useful when classes are imbalanced.

```
F1 = 2 * (Precision * Recall) / (Precision + Recall)
```

■ *Note: Use F1 Score when your dataset has imbalanced classes (e.g., 95% negative, 5% positive).*

Section 6 | Overfitting vs Underfitting

One of the most important concepts in ML — understanding how well your model generalizes to new, unseen data.

Underfitting	Good Fit	Overfitting
Model too simple	Model is just right	Model too complex
High training error	Low train + test error	Low train, high test error
High Bias	**Low Bias + Low Variance**	**High Variance**
Learns too little	*Learns just enough*	*Memorizes training data*

Simple Real-World Examples

Underfitting Example:

A student who barely studied and gives vague answers to all questions. They can't even answer simple questions correctly — too little learning.

Overfitting Example:

A student who memorized the textbook word-for-word but cannot answer any question that is worded differently. They know the training data, not the concept.

Good Fit Example:

A student who understood the concepts and can answer both familiar and new questions correctly. That's the goal!

How to Fix Overfitting

- **Get more training data** — the most effective solution
- **Reduce model complexity** — use fewer features or shallower trees
- **Regularization** — adds a penalty for overly complex models (L1/L2 Regularization)
- **Dropout** — randomly disables neurons during training (for neural networks)
- **Cross-Validation** — use k-fold CV to better estimate real performance
- **Early Stopping** — stop training when validation error starts increasing

How to Fix Underfitting

- **Use a more complex model** — add more layers or features

- **Train for longer** — more epochs / iterations
- **Add more relevant features** — feature engineering
- **Reduce regularization** — if it's too strong, it causes underfitting

> ■ *Tip: Always check both training accuracy and test accuracy. A big gap = overfitting.*

Section 7 | Basic Deep Learning Intro

Deep Learning is a subset of Machine Learning inspired by the structure of the human brain. It uses **Neural Networks** to learn complex patterns from large datasets.

What is a Neural Network?

A Neural Network is made of layers of connected nodes called **neurons**. Each neuron takes some input, processes it, and passes output to the next layer.

Layer	Role	Simple Analogy
Input Layer	Receives raw data	Your eyes receiving visual information
Hidden Layer(s)	Learns abstract features from the data	Your brain processing what you see
Output Layer	Produces the final prediction	Your brain deciding: 'that is a cat'

How a Neuron Works (Simple Intuition)

- **1. Receive inputs** — e.g., pixel values of an image
- **2. Multiply by weights** — some inputs matter more than others
- **3. Add a bias** — shifts the output
- **4. Apply activation function** — decides if the neuron 'fires'
- **5. Pass output** — to the next layer of neurons

■ *Note: Weights are automatically adjusted during training through a process called Backpropagation.*

Common Activation Functions

- **ReLU** (Rectified Linear Unit): max(0, x) — most popular for hidden layers
- **Sigmoid**: Outputs 0–1 — used in output layer for binary classification
- **Softmax**: Outputs probabilities for multiple classes — used for multi-class output

Training a Neural Network

- **Forward Pass**: Input flows through the network to produce a prediction

- **Loss Calculation**: Measure how wrong the prediction is
- **Backward Pass (Backpropagation)**: Error flows backward to update weights
- **Gradient Descent**: Optimization method to minimize the loss function
- **Epochs**: One full pass through the entire training dataset

When to Use Deep Learning

• Large amounts of data available (1000s to millions of samples)
• Complex tasks: image recognition, speech, NLP
• Regular ML algorithms are not performing well enough
• Sufficient compute resources (GPU recommended)

Deep Learning vs Traditional ML

Aspect	Traditional ML	Deep Learning
Data Needed	Small to medium	Large (thousands+)
Feature Engineering	Manual (required)	Automatic
Interpretability	High	Low (black box)
Training Speed	Fast	Slow (needs GPU)
Performance	Good for tabular data	Excellent for images/text/audio

■ *Tip: For structured/tabular data (like CSV files), traditional ML often outperforms Deep Learning.*